

Cross-platform development



Flutter

Introduction

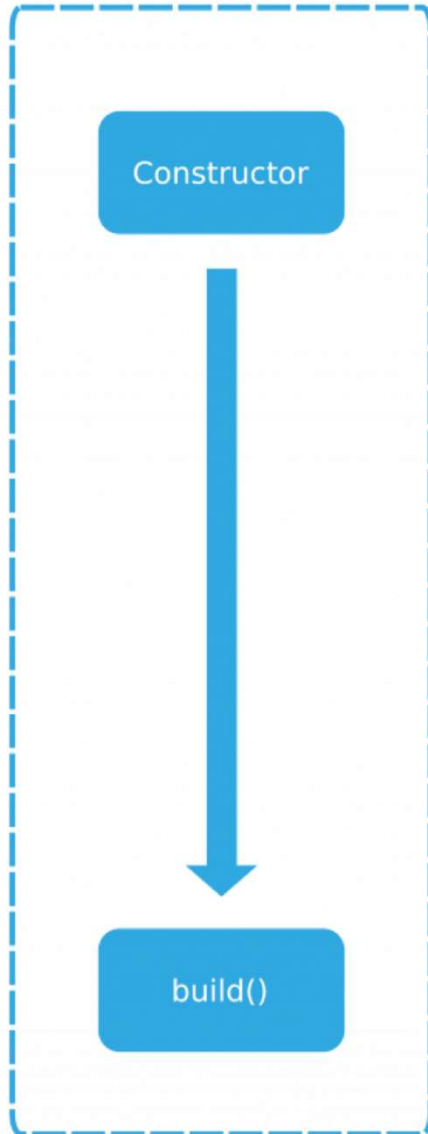
Refer to the theoretical slides for information about the Dart language and additional information about Flutter

Widgets

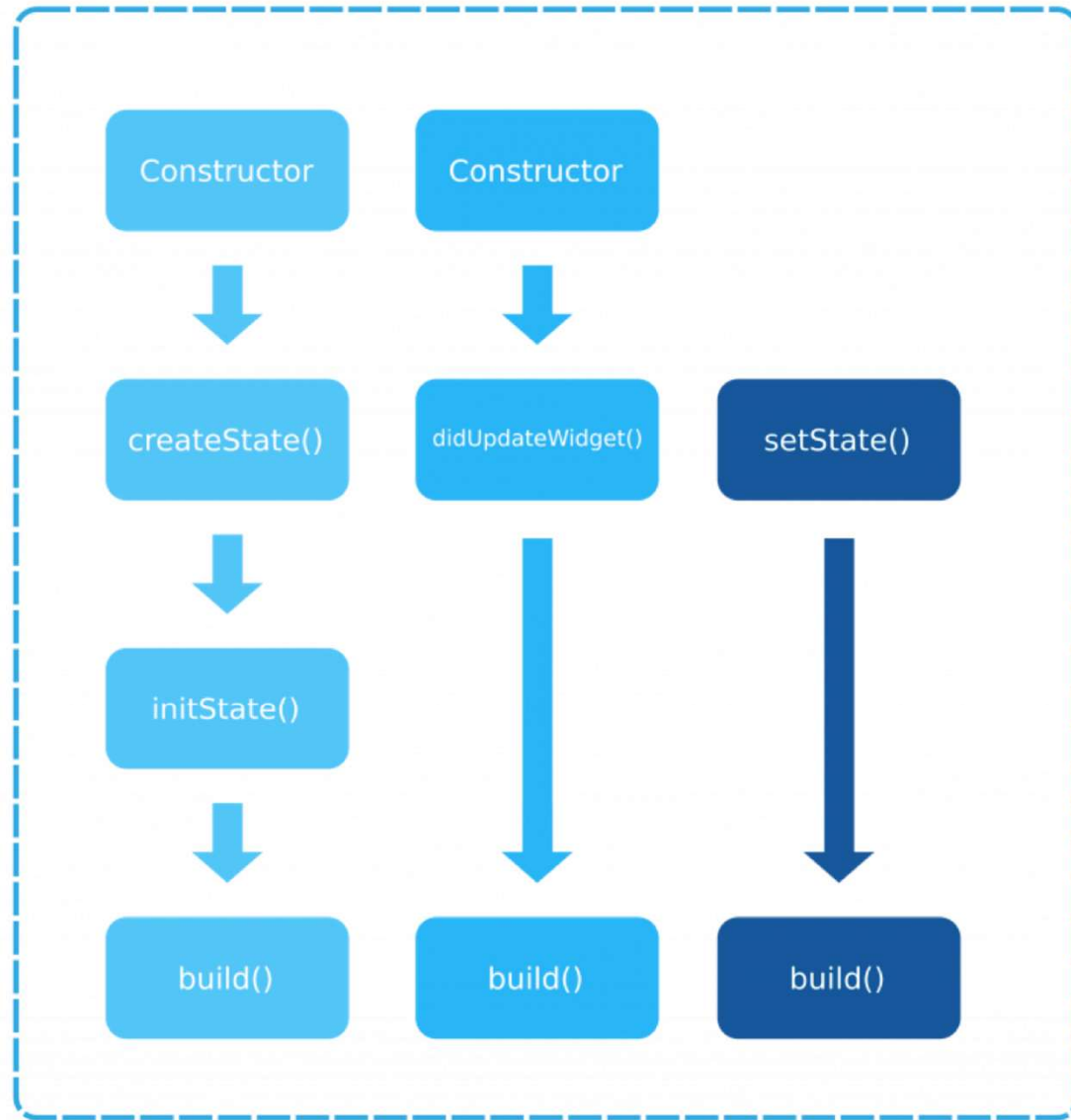
- The UI is described through a list of components referred to as Widgets
- *Stateless vs. Stateful Widgets*
 - StatelessWidget
 - Always built from scratch, when the parent *widget invokes the* `build()` method for its interface to be created
 - StatefulWidget
 - Return a `State<T>` object when instantiated and which is maintained throughout the life cycle of that *widget*
 - It is in the context of the *state object that the* `build()` method is defined so that its interface is created
 - *The lifecycle of the widget can be "followed" (to include some special processing during the evolution in the object lifecycle) overriding several methods that are available in the widget*

Widgets

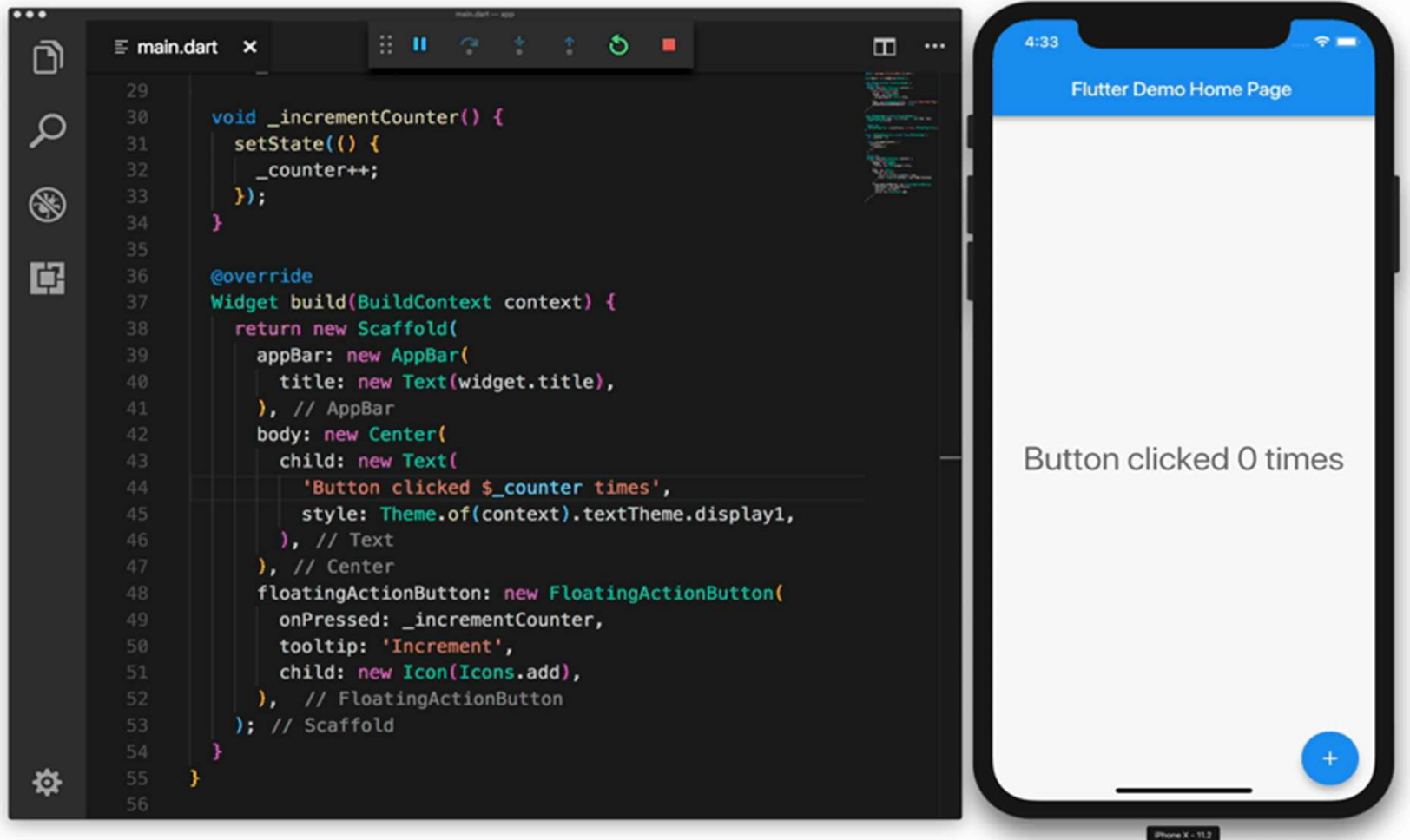
Stateless



Stateful

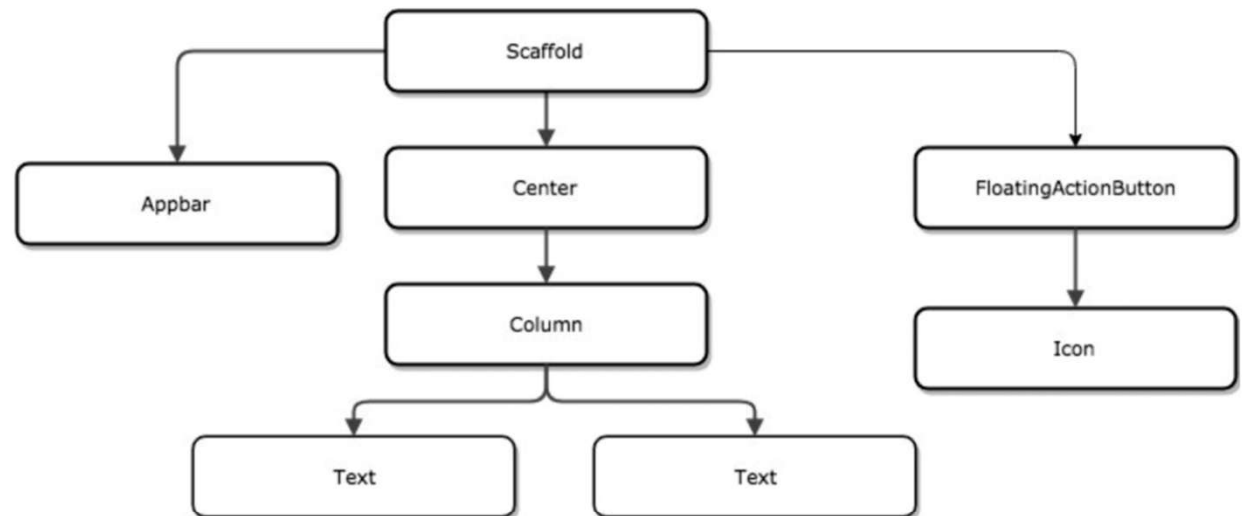
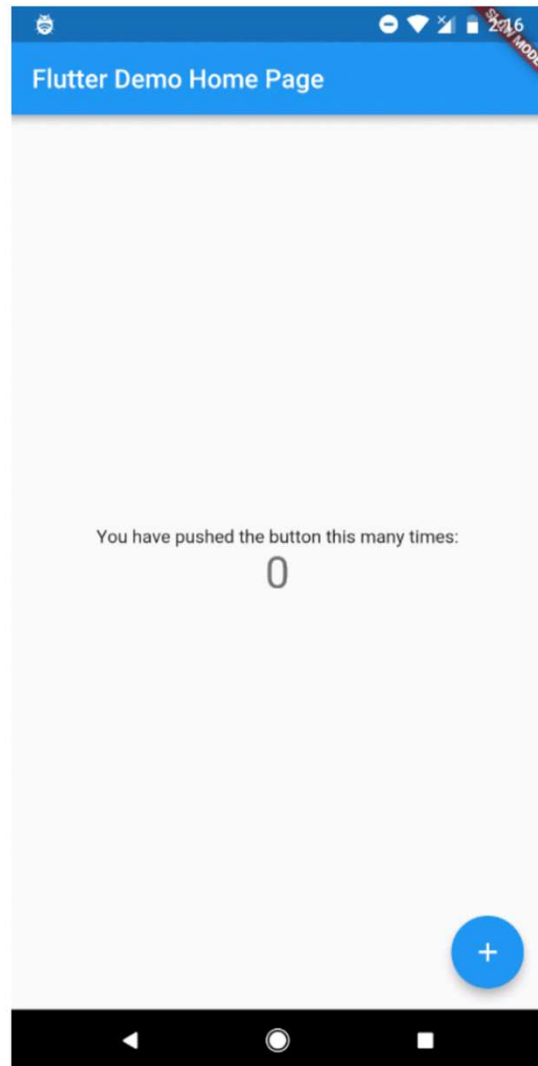


Widget tree and hot reload

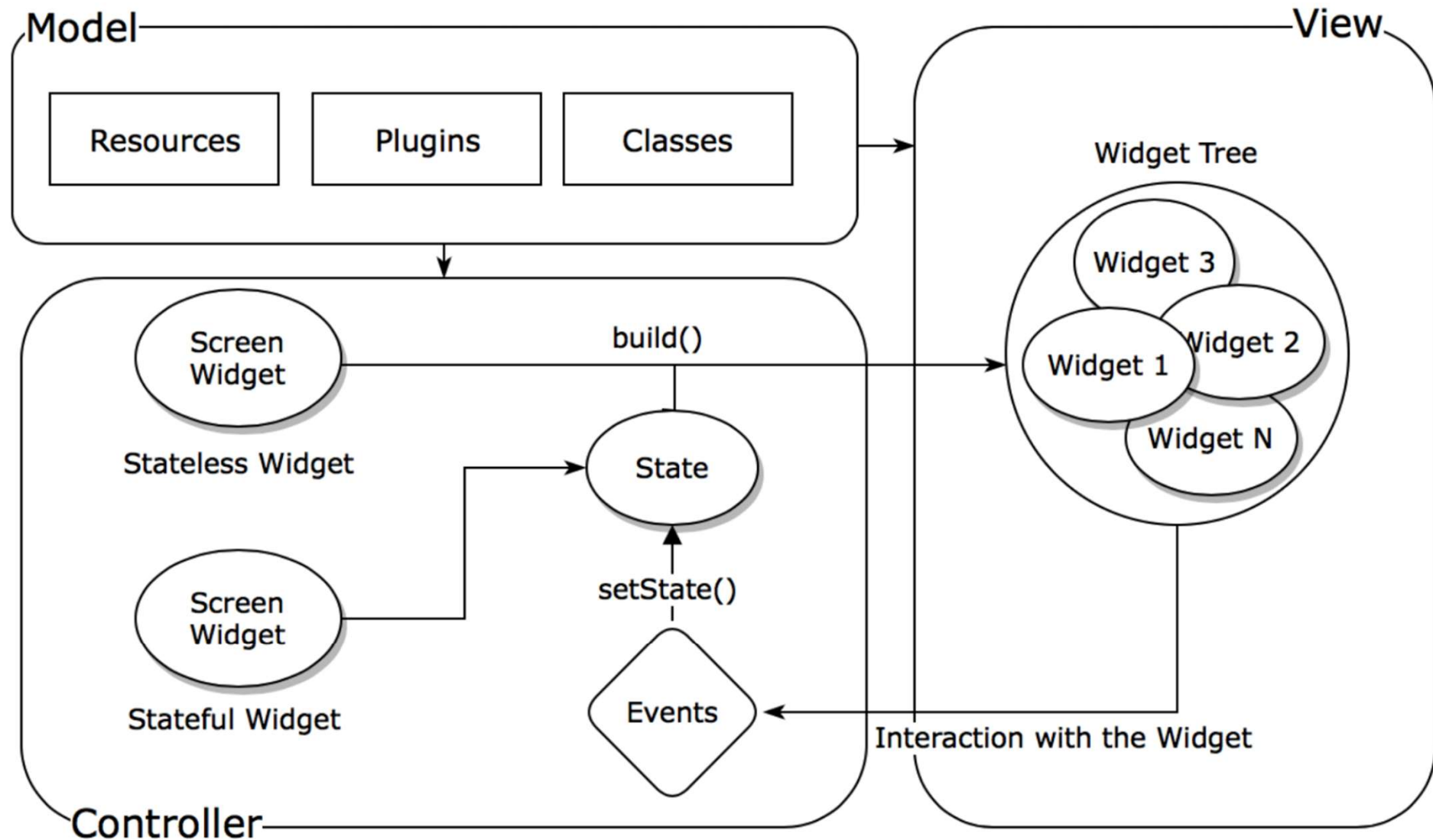


Widget tree

Widget tree



State management



State management

- Whenever a UI change/update is desired, the `setState ( () {} )` method must be invoked
- Changed variables related to the state must be placed within `setState`

```
setState ( () { _myState = newValue; } );
```
- Asynchronous calls should not be made within `setState ( () {} )`
 - If the state is updated after an asynchronous call, it can be called later without values

Installation – Flutter SDK

- Access:
 - <https://docs.flutter.dev/get-started/install>
- Select the desired OS
- Follow the instructions for each platform
 - Update the 'path' environment variable to include the 'bin' folder of the *Flutter* installation
- Run the `flutter doctor -v` command to verify the installation

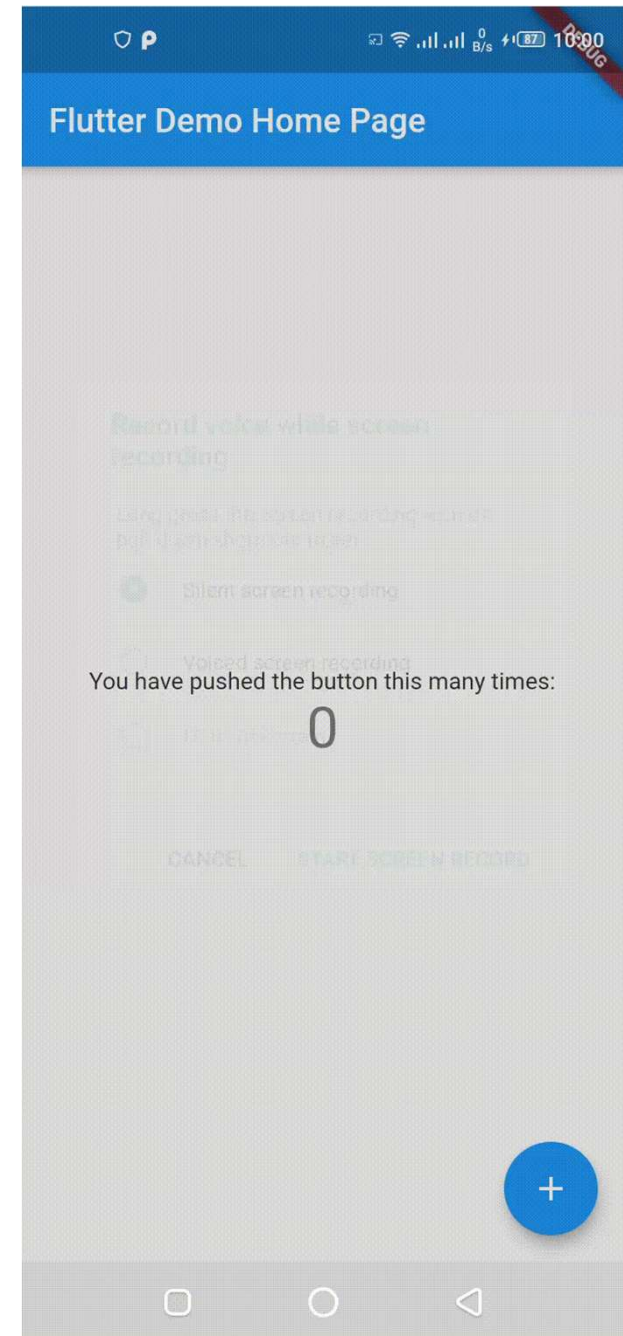
Installation – IDE

- Various IDEs can be used to develop applications in Flutter
- Most used environments
 - Visual Studio Code
 - Android Studio
 - IntelliJ
- Steps:
 - Access the extension/*plugin* library
 - Search for Flutter
 - Install the extension/*plugin*

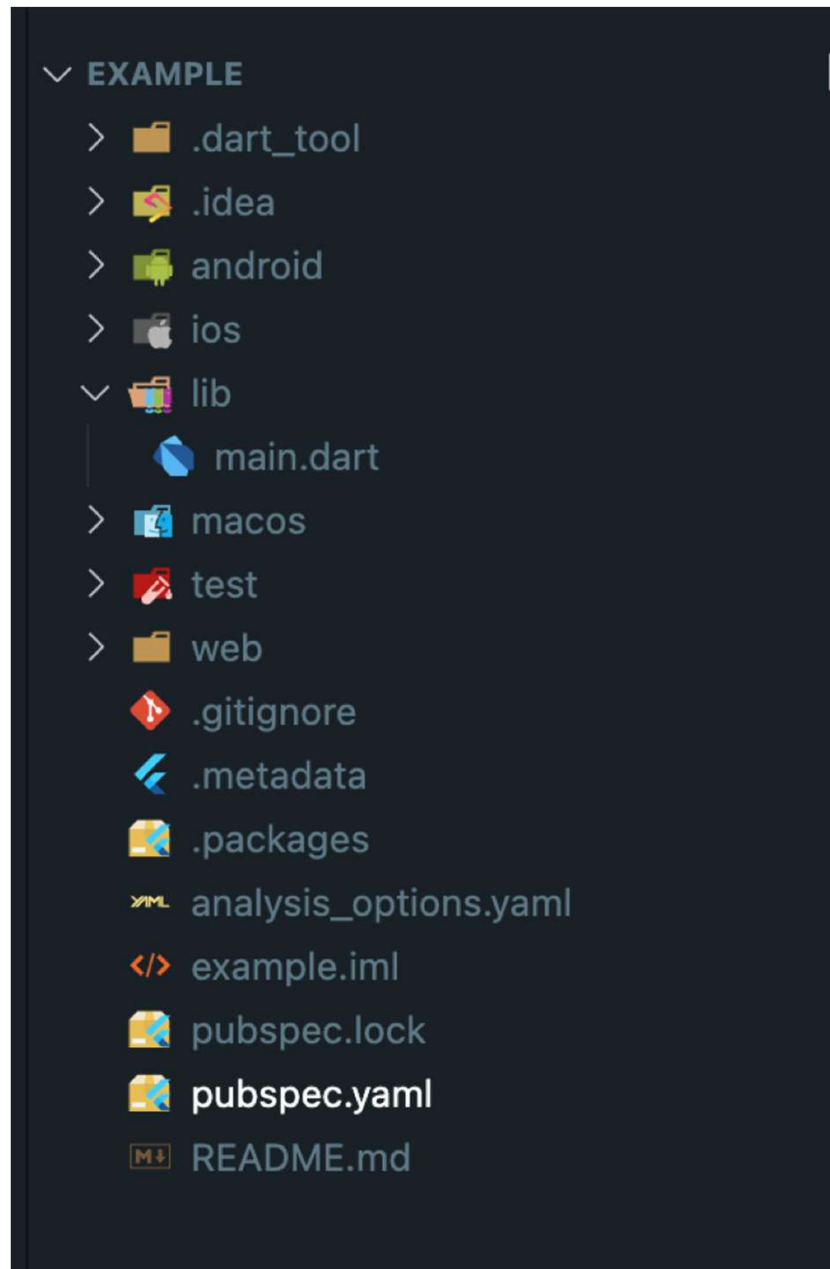


Project creation

- Android Studio or IntelliJ
 - New project → Flutter
- Visual Studio Code
 - Open the command line
 - Execute:
`flutter create example`
- An example project is created with a button that allows to increase a value

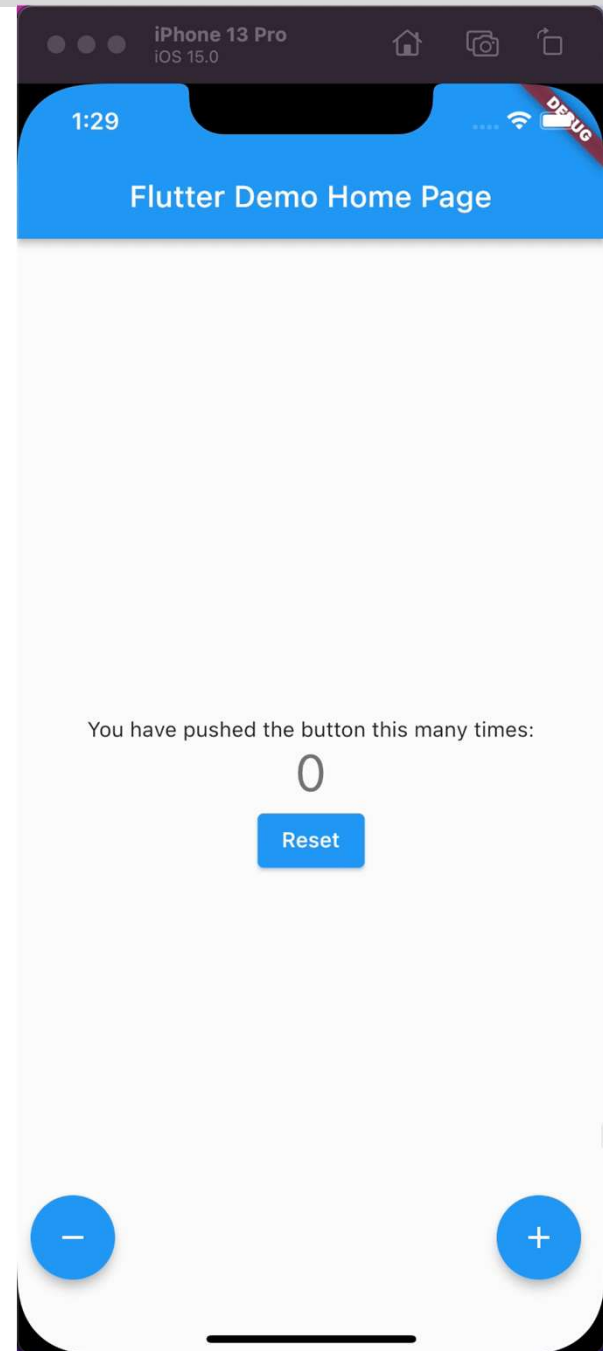


Structure of a project



Challenge

- In addition to changing of the counter, the background color will change to a random color
- Add a button to decrement and another to *reset*
- When reaching the value 10, show the *widget* FlutterLogo





Flutter

Navigation

Asynchronous API and REST API

Navigation

- Navigation between screens in *Flutter* can be done using the object: `Navigator`
- It can be done...
 - explicitly
 - *An instance of the widget* that implements the new screen is indicated
 - through named routes
 - Routes can be previously defined in `MaterialApp`
 - Mapping between a name and a *widget* that implements the new screen (using a *builder*)

Navigation

- Navigation must be carried out in the context of the UI, requiring access to the `BuildContext` to contextualize the next screen
 - `BuildContext` object is received as a parameter of the `build` method
 - Available in `State` classes through the `context` property
- Alternatively, a `Key<NavigatorState>` associated with the the field `navigatorKey` of the `MaterialApp` can be used, which will allow navigation without using the `BuildContext`

Navigation

- The Navigator object offers an API for navigating between screens
 - To use the API through named routes, the methods with the "named" suffix should be used
- Some more common examples:
 - `push/pushNamed`
 - Adds the new screen on top
 - `replace/replaceNamed`
 - Adds the new screen, disposing the previous one (using `dispose`)
 - `pop`
 - Disposes the current screen widget , returning to the previous one
 - `popUntil`
 - Disposes the current widget screen until the callback `predicate` returns `true`
 - `pushAndRemoveUntil/pushNamedAndRemoveUntil`
 - Adds a new screen, disposing of all previous ones until the callback `predicate` returns `true`

Explicit navigation

- The `push` method is used in the explicit navigation, passing an object of type `Route`
 - The `Route` object will be responsible for defining how the transition from screen A to B will be made, as well as its *settings*

```
void navigateUsingExplicitNavigation(){  
    Navigator.of(context).push(MaterialPageRoute(builder: (_) =>  
        MyScreenWidget()));  
}
```

Navigation with route names

- When navigating with route names, the routes available are defined in the app's base widget (for example, `MaterialApp`) and then specifying the route name and arguments in the `Navigator` navigation methods.

- Route definition:

```
@override
Widget build(BuildContext context) {
  return MaterialApp(
    title: 'Flutter Demo',
    initialRoute: 'myRouteName',
    routes: {
      'myRouteName': (BuildContext context) => Container(),
    },
  );
}
```

- To navigate to another screen

```
void navigateUsingNamedNavigation(){
  Navigator.of(context).pushNamed('myRouteName');
}
```

Exercise 1

- Continuing with the application of the previous example...
 - Add a button for explicit navigation to a new screen, called `SecondScreen`
 - Create the second screen first as `StatelessWidget` and then convert to `StatefulWidget`
 - Add a new button to navigate using named routes

Parameter passing

- In explicit navigation, when using the `Navigator` parameters can be passed between screens, through the `optional settings` property
 - In this property a `RouteSettings` object is indicated, where parameters will be passed (in addition to other attributes)

```
void navigateUsingExplicitNavigation(){  
    Navigator.of(context).push(  
        MaterialPageRoute(  
            builder: (_) => MyScreenWidget(),  
            settings: RouteSettings(arguments: 'Hello world'),  
        ),  
    );  
}
```

Parameter passing

- When navigating with route names, when using the `Navigator` object parameters can be passed between screens, using the optional `arguments` property

```
void navigateUsingNamedRouteNavigation(){  
    Navigator.of(context).pushNamed('myRouteName', arguments:  
MyComplexObject());  
}
```

Parameter passing

- To access the parameters, use the object `ModalRoute`

```
Object myObj = ModalRoute.of(context)?.settings.arguments;
```

- It is necessary to *cast* to the desired type, since it can be any `Object`?

Asynchronous API

- When it is necessary to perform a task asynchronously, the function where this task is performed must be tagged with the *keyword* of Dart `async`
 - `async` functions return a `Future<T>` object
 - In the context of this task, calls to blocking functions can be made
 - To wait for the task completion, a call must be made with the keyword `await`

Exercise 2

- Continuing with the application of the previous exercise...
 - When navigating to the second screen
 - Pass the counter information to the second screen and display it
 - Place a button to go back and return the counter value multiplied by 2
 - Use an asynchronous operation on the first screen to receive the value in question